

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

Name: P. Harika

Register No: 192311243

P. Harika

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: Total Marks: 20

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: -150

Connection: keep-alive

Incorrect or Invalid Fields in the Header

The response contains one invalid field:

- **Content-Length: -150**
 - Content-Length cannot be negative.
 - It must be a **non-negative integer** representing the size of the response body in bytes.

The other fields are valid:

- HTTP/1.1 200 OK ✓
- Date ✓
- Content-Type: text/html ✓
- Connection: keep-alive ✓

2. Analyze the issue with `Content-Length` and explain its impact.

Problem

`Content-Length: -150` is invalid because the HTTP specification requires it to be **0 or a positive integer**. **Impact**

- The browser may reject the response.
- The client may fail to determine where the response body ends.
- Some browsers or proxies may close the connection or display an error.
- The webpage may load incompletely or fail to load.

3. Determine whether the status code matches the response correctness.

No.

- **Status Code:** 200 OK means the request was processed successfully.
- However, the response header contains an invalid `Content-Length` value.
- Therefore, although the server indicates success, the response itself is **not correctly formatted**.

4. Suggest corrections for the identified errors.

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>TestPage</title>
  </head>
  <body>
    <div>
      <p>HelloWorld
    </div>
  </body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.

The given HTML has the following errors:

- The `<p>` tag is not closed (`</p>` is missing).
- The `<div>` is closed before the `<p>` tag, resulting in improper nesting.
- The HTML should follow proper opening and closing tag order.

2. Analyze how the browser will interpret this structure.

Modern browsers automatically correct malformed HTML while parsing the document.

- The browser will automatically insert the missing `</p>` tag before the closing `</div>`.
- The DOM tree will be adjusted so that the page is displayed correctly.
 - Different browsers may handle the correction slightly differently, which can lead to inconsistent rendering.

3. Determine the impact on rendering and layout.

- The page may still display **"Hello World"** correctly because browsers attempt to fix HTML errors.
- Improper HTML structure can cause inconsistent layouts across different browsers.
- CSS styles and JavaScript DOM manipulation may not behave as expected.
- Invalid HTML reduces code readability and maintainability.

4. Suggest corrections to make the DOM valid.

```
<!DOCTYPE html>
<html>
<head>
  <title>Test Page</title>
</head>
<body>
  <div>
    <p>Hello World</p>
  </div>
</body>
</html>
```

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and

POST. **GET**

- Sends data in the URL as query parameters.
- Data is visible in the browser address bar.
- Used to retrieve data from the server.
- Limited amount of data can be sent.
- Less secure for sensitive information.

POST

Sends data in the HTTP request body.

Data is not visible in the URL.

Used to submit or send data to the server.

Can send larger amounts of data.

2. Analyze which method is more suitable for login and why.

The **POST** method is more suitable for login because:

- User credentials are sent in the request body instead of the URL.
- Passwords are not exposed in the browser address bar, history, or bookmarks.
- It provides better security when combined with **HTTPS**.
- It is the standard method used for authentication forms.

3. Identify security risks in using GET for authentication.

Using **GET** for login has the following risks:

- Username and password are visible in the URL.
- Credentials may be stored in browser history.
- Sensitive data can be saved in server logs.
- URLs can be bookmarked or shared accidentally.
- Credentials may be exposed if the URL is intercepted.

4. Suggest best practices for secure login implementation.

Use the **POST** method for login forms.

Always use **HTTPS** to encrypt data during transmission.

Store passwords securely using strong hashing algorithms (e.g., bcrypt or Argon2), never in plain text.

Validate user input on both the client and server.

Implement CSRF protection for login forms.

Use secure session management and session timeouts.

Enforce strong password policies and consider multi-factor authentication (MFA).

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

Questions:

1. Identify missing closing tags.

The HTML code has the following missing closing tags:

- `</h1>` is missing.
- `</p>` is missing.
- The `<img>` tag should include the required `alt` attribute for accessibility.

2. Analyze issues with the `<img>` tag.

The `<img>` tag has the following issue:

- It does not have an **`alt` attribute**, which is required for accessibility and provides alternative text if the image cannot be displayed.
- Although `<img>` is a **void element** (it does not require a closing tag), it should be written with an `alt` attribute.

Example:

```

```

3. Determine how browsers will handle these errors.

- Browsers automatically insert the missing `</h1>` and `</p>` tags while parsing the HTML.
- The page will usually display correctly, but different browsers may render it slightly differently.
- If the image cannot be loaded, no descriptive text will be shown because the `alt` attribute is missing.
- Invalid HTML may affect CSS styling, JavaScript behavior, and accessibility.

4. Suggest a corrected version of the code.

```
<!DOCTYPE html>
<html>
<head>
  <title>Sample</title>
</head>
<body>

  <h1>Welcome</h1>

  <p>This is a paragraph.</p>

  
</body>
</html>
```

Result:

This corrected version is valid HTML, properly closes all required tags, includes the `alt` attribute for the image, and ensures consistent rendering across browsers.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.

1. Interpret each step in the communication flow.

1. User enters a URL in the browser

- The user types a website address (URL) into the browser.

2. DNS lookup occurs

- The browser contacts a DNS server to convert the domain name into an IP address.

3. HTTP request is sent

- The browser sends an HTTP request to the web server using the IP address.

4. Server processes the request

- The server receives the request, processes it, accesses databases or files if needed, and prepares the response.

5. Response is returned

- The server sends an HTTP response containing the requested webpage and status code (e.g., 200 OK).

6. Browser renders the page

- The browser interprets the HTML, CSS, and JavaScript and displays the webpage to the user.

2. Identify where delays can occur.

Delays may occur at the following stages:

- **DNS lookup** – Slow DNS resolution.
- **Network latency** – Slow internet connection.
- **HTTP request/response** – Network congestion or packet loss.
- **Server processing** – Heavy server load or slow database queries.
- **Browser rendering** – Large images, CSS, or JavaScript files.

3. Analyze the role of DNS in the process.

- DNS (**Domain Name System**) translates the website's domain name (e.g., www.example.com) into its corresponding IP address.
- This allows the browser to locate and connect to the correct web server.
- Without DNS, users would need to remember numerical IP addresses instead of domain names.

4. Suggest optimizations to improve performance.

- Use **DNS caching** to reduce lookup time.
- Enable **browser caching** for frequently accessed resources.
- Compress files using **Gzip** or **Brotli**.
- Optimize and compress images.
- Minify HTML, CSS, and JavaScript files.
- Use a **Content Delivery Network (CDN)** to serve content from nearby servers.
- Enable **HTTP/2** or **HTTP/3** for faster communication.
- Optimize server-side code and database queries.
- Use **HTTPS** with keep-alive connections for efficient communication.

Conclusion:

The browser-server communication process involves DNS resolution, HTTP request/response, server processing, and browser rendering. Performance can be improved through DNS caching, optimized resources, compression, CDNs, and efficient server configuration.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data

Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand